

UXL SKILLS EVALUATOR

oneDNN Skill

Primitives, memory layouts, fusion, and benchdnn evidence

September 2026 • 5 of 6 evaluation tasks implemented

Why this is useful to oneDNN at first glance

- **Protect the semantic boundary** — Agents must state shapes, strides, data types, attributes, layout assumptions, and tolerances before tuning.
- **Prevent reorder-driven regressions** — The skill treats physical layout and reorder placement as part of design, not cleanup.
- **Make fusion claims testable** — Build unfused parity first, then add post-ops or graph fusion and re-check.
- **Use project-native evidence** — benchdnn modes, verbose implementation selection, existing tests, and representative shapes anchor the workflow.

What the skill tells an agent today

- **Choose primitives versus graph deliberately** — Prefer explicit primitives for isolated control; use graph/fusion only after individual parity.
- **Construct memory descriptors explicitly** — Logical shape is not physical layout; optimized formats require intentional boundaries.
- **Classify benchdnn failures** — Separate create, reference-data, execute, compare, and report phases before editing code.
- **Gate performance on correctness** — Record engine, mode, descriptor, selected implementation, baseline, and measurement environment.
- **Verify current backend support** — No static promise that a primitive or device path exists.

Evaluation inventory: 5 implemented, 1 honest gap

- **Implemented** — Matmul memory descriptors; convolution fusion parity; extra reorder regression; framework blocked layout; benchdnn no-reference-memory incident.
- **Incident evidence** — benchdnn-no-ref-memory is sourced from upstream issue #5732 and its accepted repair.
- **Planned** — A real backend-unimplemented-primitive failure on a declared target device.
- **What this proves** — The implemented tasks check workflow and repair behavior; they do not qualify every backend or primitive.

How we chose the content

- **Official oneDNN sources** — Project docs, examples, tests, benchdnn behavior, releases, and ownership files.
- **Failure-shaped scenarios** — Layout mismatches, unnecessary reorders, fusion parity, framework boundaries, and harness-mode bugs.
- **A review standard** — Realistic prompts, deterministic correctness checks, limitations, and current-source verification.
- **Deliberate gaps** — Target-dependent support remains planned until a real environment and reproducer exist.

What project ownership would look like

- **Review a small surface** — SKILL.md, official source ledger, public skill card, eval prompts, and the oneDNN task slice.
- **Change it through normal PRs** — Correct terminology, narrow triggers, add a gotcha, refresh sources, or retire an evaluation.
- **UXL keeps the machinery** — Common validators, Harbor execution, dashboards, and hardware-runner contracts stay shared.
- **Promotion remains project-gated** — No reviewed/project-owned label without a maintainer decision and recorded limitations.

A focused 30-minute maintainer review

- **1. Confirm scope** — Are primitive/graph choices, layout boundaries, fusion, numerics, verbose output, and benchdnn the right first slice?
- **2. Correct three things** — One trigger, one gotcha, and one unsupported/overstated assumption.
- **3. Judge the six tasks** — Keep, rewrite, or remove each; nominate a better target-device failure if available.
- **4. Name an owner** — One person or team for release-time or quarterly source review.
- **Outcome** — A project-backed skill that gives coding agents the same first-pass discipline maintainers expect.